

Numerical Coordinate Regression with Convolutional Neural Networks

Aiden Nibali Zhen He Stuart Morgan Luke Prendergast
La Trobe University, Australia

Abstract

We study deep learning approaches to inferring numerical coordinates for points of interest in an input image. Existing convolutional neural network-based solutions to this problem either take a heatmap matching approach or regress to coordinates with a fully connected output layer. Neither of these approaches is ideal, since the former is not entirely differentiable, and the latter lacks inherent spatial generalization. We propose our **differentiable spatial to numerical transform (DSNT)** to fill this gap. The DSNT layer adds no trainable parameters, is fully differentiable, and exhibits good spatial generalization. Unlike heatmap matching, DSNT works well with low heatmap resolutions, so it can be dropped in as an output layer for a wide range of existing fully convolutional architectures. Consequently, DSNT offers a better trade-off between inference speed and prediction accuracy compared to existing techniques. When used to replace the popular heatmap matching approach used in almost all state-of-the-art methods for pose estimation, DSNT gives better prediction accuracy for all model architectures tested.

1. Introduction

In recent years, deep convolutional neural networks (CNNs) have proven to be highly effective general models for a multitude of computer vision problems [1, 2, 3, 4]. One such problem is *coordinate regression*, where the goal is to predict a fixed number of location coordinates corresponding to points of interest in an input image. A well-known instance of this problem is human pose estimation, for which CNNs are state-of-the-art. In this paper we study CNN-based solutions to coordinate regression, using the single-person pose estimation task as an exemplar. Such solutions may exhibit the desirable properties of spatial generalization and/or end-to-end differentiability.

Spatial generalization is the ability of a model to generalize knowledge obtained at one location during training to another at inference time. If a spatially generalizable model observes a tennis ball in the top-left of an image during training, it should be able to successfully locate a similar tennis ball at a previously unseen location in a new im-

age (e.g. the bottom right). It follows that this property will make a positive contribution to the overall generalization of a coordinate regression model, since the goal is to find items anywhere in the image. In general, the success of CNNs is understood to be a result of the high generalization ability afforded by spatially shared parameters [5]. To maximize this advantage, care must be taken to avoid trainable layers which can overfit on global structure. Lin et al. [6] note that “fully connected layers are prone to overfitting, thus hampering the generalization ability of the overall network”.

An end-to-end differentiable model can be composed with other differentiable layers to form a larger model without losing the ability to train using backpropagation [7]. In the case of coordinate regression, being end-to-end differentiable means being able to propagate gradients all the way from the output numerical coordinates to the input image. It is possible to train a coordinate regression model without this property, such as by matching predicted heatmaps to target heatmaps generated from the ground truth locations. However, this approach cannot be used in architectures where the numerical coordinates are learned implicitly as intermediate values, including the prominent example of Spatial Transformer Networks [8].

There are many CNN-based solutions to other computer vision tasks, such as classification and semantic segmentation, which exhibit both spatial generalization and end-to-end differentiability. However, existing solutions for coordinate regression sacrifice one property or the other.

The most successful existing coordinate regression approach is to apply a loss directly to output heatmaps rather than numerical coordinates [9, 4, 10]. Synthetic heatmaps are generated for each training example by rendering a spherical 2D Gaussian centered on the ground truth coordinates. The model is trained to produce output images which resemble the synthetic heatmaps using mean-square-error loss. During inference, numerical coordinates are obtained from the model’s output by computing the argmax of pixel values, which is a non-differentiable operation. Although this approach has good spatial generalization, it does have a few disadvantages. Most notably, gradient flow begins at the heatmap rather than the numerical coordinates (Figure 1a). This leads to a disconnect between the loss function being optimized (similarity between heatmaps) and the metric we

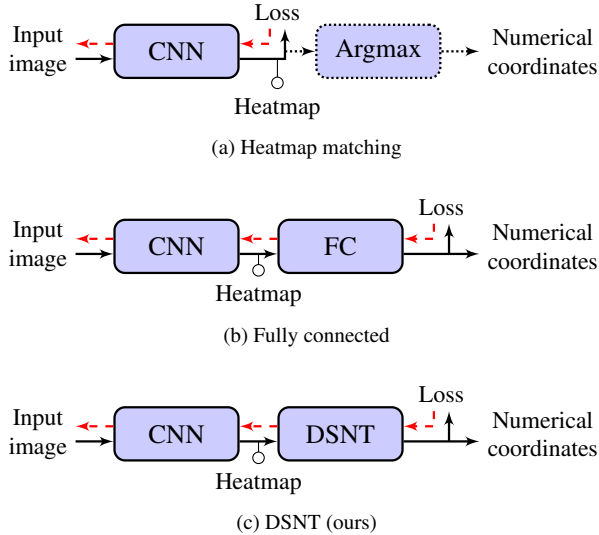


Figure 1: Comparison of coordinate regression model architectures. The arrows indicate inference (black) and gradient flow (dashed red).

are actually interested in (the distance between predicted coordinates and ground truth). Only the brightest pixel is used to calculate numerical coordinates at inference time, but all of the pixels contribute to the loss during training. Making predictions based on the argmax also introduces quantization issues, since the coordinates have their precision tied to the heatmap’s resolution.

Another coordinate regression approach is to add a fully connected layer which produces numerical coordinates [11, 8]. An attractive (and sometimes *required*) property of this approach is that it is possible to backpropagate all the way from the predicted numerical coordinates to the input image. However, the weights of the fully-connected layer are highly dependent on the spatial distribution of the inputs during training. To illustrate this point, consider an extreme situation where the training set consists entirely of coordinates located within the left-hand half of the image. Many of the fully connected layer’s input activations will be useless, and as a result weights corresponding to the right-hand side of the image will not be trained properly. So although the convolutional part of the model is spatially invariant, the model as a whole will not generalize well to objects on the right-hand side of the image. This is an inefficient usage of the training data, and causes particularly bad performance on small datasets.

We propose our *differentiable spatial to numerical transform* (DSNT) layer as an alternative to existing approaches. The DSNT layer may be used to adapt existing CNN architectures, such as a pretrained ResNet [12], to coordinate regression problems. Our technique fully preserves the spatial generalization and end-to-end differentiability of the model, without introducing additional parameters. Figure 1 illus-

	HM	FC	DSNT
Fully differentiable	✗	✓	✓
Spatially generalizable	✓	✗	✓
No parameters	✓	✗	✓
Good for high-res output	✓	✗	✓
Good for low-res output	✗	✓	✓
Direct coordinate loss	✗	✓	✓

Table 1: Presence of desirable properties in heatmap matching (HM), fully connected output (FC), and differentiable spatial to numerical transform (DSNT).

trates how the DSNT layer fits into the model as a whole in comparison to fully connected and heatmap matching approaches. Table 1 summarizes the features that DSNT poses which selectively appear in fully connected (FC) and heatmap matching (HM) based approaches.

We find that DSNT is able to consistently outperform the accuracy of heatmap matching and fully connected approaches across a variety of architectures on the MPII human pose dataset [13], and is therefore a suitable replacement in most situations. Our experiments show that state-of-the-art stacked hourglass models [4] achieve higher accuracy when heatmap matching is replaced with DSNT. For ResNet-34 models, DSNT outperforms heatmap matching by 90.5% with 7×7 pixel heatmaps, and by 2.0% with 56×56 pixel heatmaps. Since accuracy at low heatmap resolution is much better with DSNT, a wider variety of efficient architectures may be considered for coordinate regression. For instance, a simple ResNet-50 network with DSNT is comparable in accuracy to an 8-stack hourglass network, but exhibits triple the speed and half of the memory usage during inference.

The DSNT layer presented in this paper is very similar to the soft-argmax operation of Luvizon et al. [14], which was developed in parallel with our own work. The soft-argmax has also been applied to different problem domains prior to this [15, 16]. However, we extend the idea further by proposing a regularization strategy which increases prediction accuracy. Additionally, we conduct a comprehensive set of experiments exploring configurations and properties of the operation, and the trade-off between accuracy and inference speed in the context of complete pose estimation models.

2. Related Work

Heatmap matching and fully connected layers are prevalent in existing solutions to problems including human pose estimation and Spatial Transformer Networks. As such, the following section describes how existing coordinate regression approaches are applied in those contexts. Although this paper focuses on pose estimation as an exemplar of the DSNT layer’s capability, our approach is broadly applica-

ble to any coordinate regression problem.

2.1. Human pose estimation

DeepPose [11] is one of the earliest CNN-based models to perform well on the human pose estimation task, and helped pioneer the current dominance of deep learning in this area. In order to predict pose joint locations, DeepPose uses a multi-stage cascade of CNNs with fully connected outputs. The first stage of the cascade predicts the absolute coordinates of the joint locations, and subsequent stages refine the predictions by producing relative position deltas. The authors argue that the cascade arrangement enables reasoning about human pose at a higher level, since later stages are able to analyze global structure.

Shortly after DeepPose was published, Tompson et al. [9] proposed a higher accuracy model which uses heatmap matching to calculate loss. Heatmap matching has since become overwhelmingly dominant amongst human pose estimation models, including the state-of-the-art stacked hourglass architecture [4] which is fundamental to current leaders of the MPII single person pose estimation challenge [10, 17, 18, 19]. Each “hourglass” in a stacked hourglass network uses the first half of its layers to downsample activations, and the second half to upsample back to the original size. By stacking multiple hourglasses together, the network is able to process data in a repeated bottom-up, top-down fashion, achieving an effect similar to DeepPose’s cascade. Skip layers are used extensively throughout the architecture, both within and across individual hourglasses, which makes the model easier to train with backpropagation.

Very recent research suggests that adversarial training [20] aids in the prediction of likely joint positions by having a discriminator learn the difference between coherent and nonsensical poses [17, 18]. Although we do not conduct such experiments in this paper, we observe that adversarial training is orthogonal to our findings and could be combined with our DSNT layer as future work.

2.2. Spatial Transformer Networks

The internal Localisation Network component of Spatial Transformer Networks [8] uses a fully connected layer to predict translation transformation parameters, which are effectively just 2D location coordinates. It is not possible to use heatmap matching in such a model, as gradients must be passed backwards through the coordinate calculations. In contrast, our DSNT layer could be used as a drop-in replacement for calculating the translation parameters.

3. Main idea

We introduce a new differentiable layer for adapting fully convolutional networks (FCNs) to coordinate regression. FCNs are a broad class of CNNs which rely solely on spatially invariant operations to produce their outputs [6], and are hence naturally spatially generalizable. Most

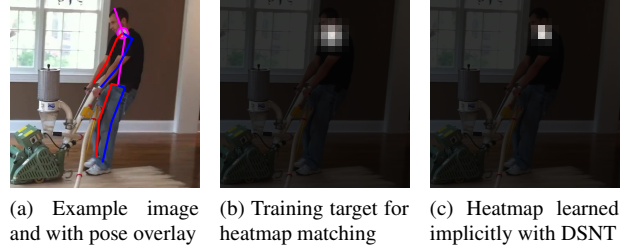


Figure 2: Spatial representations of an example neck location. Image (b) is a 2D Gaussian rendered at the ground truth location, whereas (c) is learned freely by a model.

CNNs with fully connected output layers can be converted into FCNs by simply removing the fully connected layers. FCNs are already **spatially generalizable and end-to-end differentiable**, so we design our new layer in such a way that these two desirable properties are preserved. This new layer—which we call the DSNT layer—is placed at the output of the FCN and transforms spatial heatmaps into numerical coordinates.

Activations are represented spatially throughout an FCN, which is very useful for tasks like semantic segmentation [2] where the output is intended to be spatial. However, for coordinate regression tasks like human pose estimation the output needs to be coordinate pairs. This begs the question: how do we transform spatial activations into numerical coordinates such that we can still effectively train the model?

Consider the case of locating a person’s neck in the input image. This location may be represented spatially as a heatmap (Figure 2b), and can be learned by an FCN since it is simply a single-channel image. The purpose of the DSNT layer is to transform such a heatmap into numerical coordinates, which is the form of output we require for coordinate regression. However, we have to be careful about how we approach designing the DSNT, since we want the layer to be part of an end-to-end trainable model. For example, if we simply take the location of the brightest pixel then we cannot calculate meaningful gradients during training. Therefore, we design the DSNT layer such that it is able to propagate smooth gradients back through all heatmap pixels from the numerical coordinates.

In contrast to heatmap matching techniques, **we do not require applying a loss directly to the heatmap output by the FCN to make it resemble** Figure 2b. Instead, the heatmap is learned indirectly by optimizing a loss applied to the predicted coordinates output by the model as a whole. This means that during training the heatmap will evolve to produce accurate coordinates via the DSNT layer. An example of an implicitly learned heatmap is shown in Figure 2c.

$\hat{Z}$	X	Y
0.0 0.0 0.0 0.0 0.0	-0.8 -0.4 0.0 0.4 0.8	-0.8 -0.8 -0.8 -0.8 -0.8
0.0 0.0 0.0 0.1 0.0	-0.8 -0.4 0.0 0.4 0.8	-0.4 -0.4 -0.4 -0.4 -0.4
0.0 0.0 0.1 0.6 0.1	-0.8 -0.4 0.0 0.4 0.8	0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.1 0.0	-0.8 -0.4 0.0 0.4 0.8	0.4 0.4 0.4 0.4 0.4
0.0 0.0 0.0 0.0 0.0	-0.8 -0.4 0.0 0.4 0.8	0.8 0.8 0.8 0.8 0.8

$$x = \langle \hat{Z}, X \rangle_F = \begin{pmatrix} 0.1 \times 0.0 & + & 0.1 \times 0.4 & + \\ 0.1 \times 0.0 & + & 0.6 \times 0.4 & + \\ 0.1 \times 0.0 & + & 0.1 \times 0.8 & + \\ 0.1 \times 0.0 & + & 0.1 \times 0.4 & + \end{pmatrix} = 0.4$$

$$y = \langle \hat{Z}, Y \rangle_F = \begin{pmatrix} 0.1 \times 0.0 & + & 0.1 \times -0.4 & + \\ 0.1 \times 0.0 & + & 0.6 \times 0.0 & + \\ 0.1 \times 0.0 & + & 0.1 \times 0.0 & + \\ 0.1 \times 0.0 & + & 0.1 \times 0.0 & + \end{pmatrix} = 0.0$$

Figure 3: Coordinate calculation using the differentiable spatial to numerical transform (DSNT).

4. The Differentiable Spatial to Numerical Transform

In this section we describe the technical details of our differentiable spatial to numerical transform (DSNT) layer. The DSNT layer has no trainable parameters, is fully differentiable, and generalizes spatially. Accordingly, it is possible to use our layer as part of a CNN model to enable numerical coordinate outputs without sacrificing end-to-end learning with backpropagation.

The input to the DSNT is a single-channel normalized heatmap, $\hat{Z}$, represented as an $m \times n$ matrix where m and n correspond to the heatmap resolution. By “normalized” we mean that all elements of $\hat{Z}$ are non-negative and sum to one—the same conditions which must be fulfilled by a probability distribution. Using such a normalized heatmap guarantees that predicted coordinates will always lie within the spatial extent of the heatmap itself. The unnormalized heatmap output of an FCN, Z , can be normalized by applying a heatmap activation function $\hat{Z} = \phi(Z)$. Suitable choices for $\phi(Z)$ are discussed in Section 4.1.

Let X and Y be $m \times n$ matrices, where $X_{i,j} = \frac{2j-(n+1)}{n}$ and $Y_{i,j} = \frac{2i-(m+1)}{m}$. That is, each entry of X and Y contains its own x - or y -coordinate respectively, scaled such that the top-left corner of the image is at $(-1, -1)$ and bottom-right is at $(1, 1)$.

By taking a probabilistic interpretation of $\hat{Z}$ we can represent the coordinates, $\mathbf{c}$, as a discrete bivariate random vector with mass function $p(\mathbf{c})$ defined as

$$\Pr(\mathbf{c} = [X_{i,j} \ Y_{i,j}]) = \hat{Z}_{i,j}$$

for all $i = 1 \dots m, j = 1 \dots n$.

In the heatmap matching approach to coordinate regression, the predicted numerical coordinates are analogous to

Table 2: Heatmap activation functions and their corresponding human pose estimation results.

Name	Rectification	PCKh
Softmax	$Z'_{i,j} = \exp(Z_{i,j})$	86.81%
Abs	$Z'_{i,j} = Z_{i,j} $	86.48%
ReLU	$Z'_{i,j} = \max(0, Z_{i,j})$	86.69%
Sigmoid	$Z'_{i,j} = (1 + \exp(-Z_{i,j}))^{-1}$	86.71%

the mode of $\mathbf{c}$. For the DSNT layer we instead take our prediction to be the mean of $\mathbf{c}$, denoted $\mu = \mathbb{E}[\mathbf{c}]$. Unlike the mode, the mean can a) have its derivative calculated, allowing us to backpropagate through the DSNT layer; and b) predict coordinates with sub-pixel precision. Equation 1 details how the expectation is calculated, and hence defines the DSNT operation. We use $\langle \cdot, \cdot \rangle_F$ to denote the **Frobenius inner product**, which is equivalent to taking the scalar dot product of vectorized matrices.

$$\text{DSNT}(\hat{Z}) = \mu = \left[\langle \hat{Z}, X \rangle_F \quad \langle \hat{Z}, Y \rangle_F \right] \quad (1)$$

Figure 3 illustrates the DSNT operation with an example. Notice how the symmetrical off-center values of the heatmap cancel each other out in the calculations. In practice, this property tends to cause the network to learn heatmaps which are roughly symmetrical about the predicted location.

One seemingly apparent flaw with using the mean instead of the mode is that the predicted coordinates will be affected adversely by outliers in the heatmap. However, it is important to keep in mind that the heatmap itself is learned with the objective of optimizing coordinate accuracy. Therefore, during training the model is encouraged to threshold its activations such that outliers are simply not placed in the heatmap at all. That is, the network is specifically punished for polluting the heatmap with low confidence outliers *because* they would adversely affect results, and hence the model can simply learn to avoid such situations.

4.1. Heatmap activation

As mentioned earlier, a heatmap activation function $\phi(Z)$ is required to normalize the heatmap before applying the DSNT. Here we will describe several choices for this function by decomposing the activation into two parts. Firstly, **each element of the input image Z undergoes rectification to produce a non-negative output. The rectified image Z' is then normalized using the L^1 norm** so that the elements sum to one (*i.e.* $\hat{Z} = (\sum Z'_{i,j})^{-1} Z'$).

Table 2 shows some possible options for the rectification function, along with validation set PCKh accuracy measurements on the MPII human pose dataset. These results were gathered using ResNet-34 models pretrained on ImageNet,

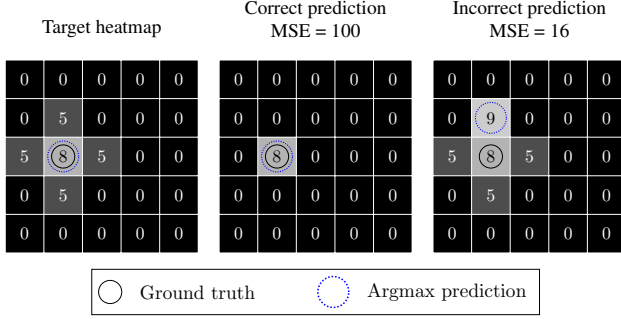


Figure 4: When heatmap matching, it is possible for predictions to worsen despite the pixel-wise MSE improving.

dilated to produce a heatmap resolution of 28×28 pixels. No regularization was used. Although the choice of rectification function does not appear to have a large impact on results, our experiments indicate that softmax works best.

5. Loss function

Since the DSNT layer outputs numerical coordinates, it is possible to directly calculate the two-dimensional Euclidean distance between the prediction μ and ground truth p . We take advantage of this fact to formulate the core term of our loss function (Equation 2).

$$\mathcal{L}_{euc}(\mu, p) = \|p - \mu\|_2 \quad (2)$$

The Euclidean loss function has the advantage of directly optimizing the metric we are interested in: the distance between the predicted and actual locations.

Contrast this with the mean-square-error (MSE) loss used in heatmap matching, which optimizes the pixel-wise similarity between the output and a synthetic heatmap generated from ground truth locations. The pixel-wise MSE loss is a much less direct way of optimizing the metric that we actually care about. During training, the model is completely ignorant of the fact that coordinate predictions are based solely on the brightest heatmap pixel. Another way to put this is that despite the Euclidean loss having a global minimum when the MSE loss is zero, we aren't guaranteed that an optimization step which improves MSE loss will improve our results. Figure 4 illustrates an example situation where improving the MSE loss degrades the predictive accuracy of the model. In this case we see that the output with a single pixel at the correct location has worse MSE but better location prediction than an almost perfectly matching heatmap with the brightest pixel placed incorrectly.

5.1. Regularization

There are many different possible heatmaps that will lead to the same coordinates being output from the DSNT layer. For example, the spread of the heatmap has no effect on

the output—blobs resembling 2D Gaussians with large variance and small variance can produce identical coordinates. Although such freedom may be viewed as beneficial, a potential drawback is that the model does not have strongly supervised pixel-wise gradients through the heatmap during training. Experimentally, we find that providing such supervision via regularization can yield marked performance improvements over vanilla DSNT.

Equation 3 shows how regularization is incorporated into the DSNT loss function. A regularization coefficient, λ , is used to set the strength of the regularizer, $\mathcal{L}_{reg}$.

$$\mathcal{L}(\hat{Z}, p) = \mathcal{L}_{euc}(\text{DSNT}(\hat{Z}), p) + \lambda \mathcal{L}_{reg}(\hat{Z}) \quad (3)$$

5.1.1 Variance regularization

By expanding upon the probabilistic interpretation of the DSNT layer (Section 4), we can **calculate the variance of coordinates**. This is described for x -coordinates in Equation 4 (y -coordinates are handled similarly). The calculated variance represents the “spread” of the blob in the heatmap, which is analogous to the size of the synthetic 2D Gaussian drawn in the heatmap matching approach.

$$\begin{aligned} \text{Var}[c_x] &= \mathbb{E}[(c_x - \mathbb{E}[c_x])^2] \\ &= \left\langle \hat{Z}, (\mathbf{X} - \mu_x) \odot (\mathbf{X} - \mu_x) \right\rangle_F \end{aligned} \quad (4)$$

We are now able to introduce a variance regularization term, Equation 5. The “spread” of the learned heatmaps is controlled by a hyperparameter, the target variance, σ_t^2 . Note that this regularization term does not directly constrain the specific shape of learned heatmaps.

$$\mathcal{L}_{var}(\hat{Z}) = (\text{Var}[c_x] - \sigma_t^2)^2 + (\text{Var}[c_y] - \sigma_t^2)^2 \quad (5)$$

5.1.2 Distribution regularization

Alternatively, we can impose even stricter regularization on the appearance of the heatmap to directly encourage a certain shape. More specifically, to force the heatmap to resemble a spherical Gaussian, we can minimize the divergence between the generated heatmap and an appropriate target normal distribution. Equation 6 defines the distribution regularization term, where $D(\cdot || \cdot)$ is a divergence measure (e.g. Jensen-Shannon divergence).

$$\mathcal{L}_D(\hat{Z}, p) = D(p(c) || \mathcal{N}(p, \sigma_t^2 \mathbf{I}_2)) \quad (6)$$

Adding a regularization term of this form is similar to incorporating the usual heatmap matching objective into the DSNT loss function.

Selecting the best regularization

In order to determine the best performing regularization term, we conducted a series of experiments on the MPII human pose dataset with a ResNet-34@28px model.

Table 3: Pose estimation results for different regularization terms, using a ResNet-34@28px model.

Regularization	λ	Validation PCKh	
		$\sigma_t = 1$	$\sigma_t = 2$
None	N/A	86.86%	
Variance	100	84.58%	85.88%
Kullback-Leibler	1	84.67%	84.15%
Jensen-Shannon	1	87.59%	86.71%

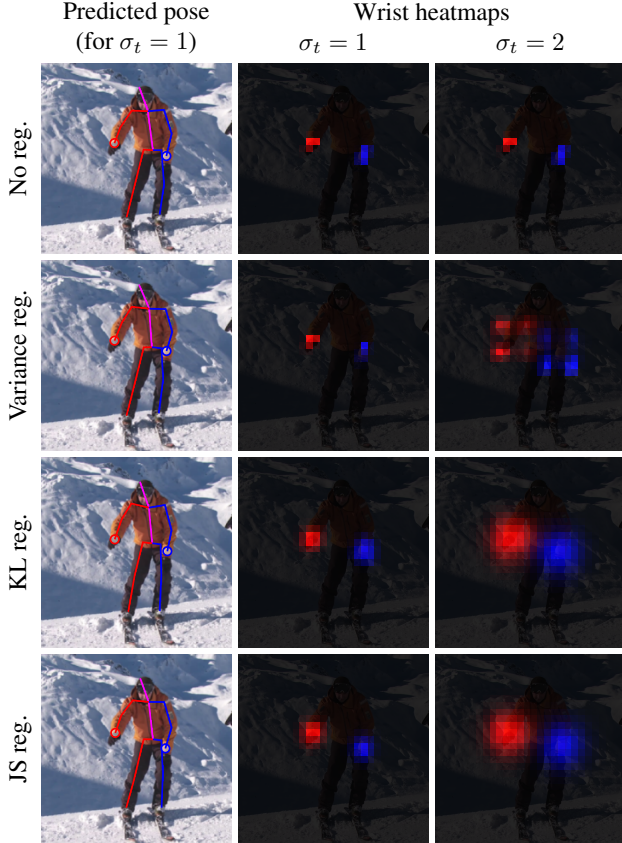


Figure 5: Heatmap appearance for models trained with different regularization terms (red = right wrist, blue = left wrist).

Firstly, we compared different options for the regularization function, $\mathcal{L}_{reg}$: variance regularization, and distribution regularization with Kullback-Leibler (KL) and Jensen-Shannon (JS) divergences. The pose estimation results in Table 3 indicate that JS distribution regularization achieves the highest accuracy. The sample heatmap images shown in Figure 5 illustrate how dramatically the choice of regularization term can change the appearance of heatmaps. For example, distribution regularization (using either KL or JS divergence) very effectively encourages the production of distinctly Gaussian-shaped blobs. In contrast, variance reg-

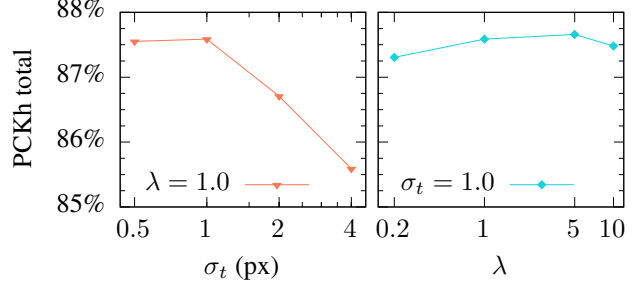


Figure 6: Varying the Gaussian size and regularization strength for JS regularization.

ularization with $\sigma_t = 2$ results in an interesting strategy of splitting the heatmap into four blobs around the joint.

We conducted further experiments to determine the optimal regularization hyperparameters (Figure 6). The accuracy of the model was found to be quite robust with respect to the regularization strength, λ (Equation 3). In terms of the target Gaussian standard deviation, σ_t , values in the range of half a pixel to one pixel were found to work well.

6. Experiments

6.1. Model base

We conducted experiments using two different fully convolutional model architectures for the CNN portion of the coordinate regression network (see Figure 1).

ResNet The ResNet architecture [12] is well-known for performing extremely well in classification tasks. We converted ImageNet-pretrained ResNet models into fully convolutional networks (FCNs) by removing the final fully connected classification layer. Such models produce 7×7 px spatial heatmap outputs. However, we were able to adjust the heatmap resolution of the FCN using dilated convolutions, as proposed by Yu and Koltun [21]. More specifically, we change the convolution stride from 2 to 1 in one or more downsampling stages, then use dilated convolutions in subsequent layers to preserve the receptive field size. For each downsampling stage modified in this way, the heatmap resolution increases by a factor of two.

Stacked hourglass The stacked hourglass architecture [4] is currently state-of-the-art for human pose estimation [10, 17, 18, 19]. The heatmap resolution of this architecture is 64×64 px.

6.2. Output strategy

Heatmap matching (HM) We follow the specific technique used by Newell et al. [4]. MSE pixel-wise loss is applied directly to the output of the FCN. During inference, numeric coordinates are calculated based on the brightest pixel of the heatmap, with small adjustments to the location made based on the brightness of adjacent pixels.

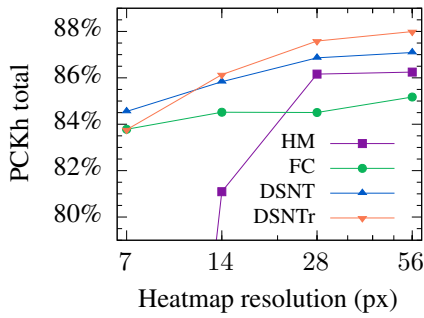


Figure 7: Varying output resolution and strategy for ResNet-34 models.

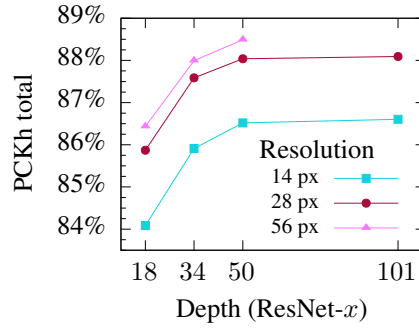


Figure 8: Varying ResNet [12] depth and heatmap resolution for DSNTTr.

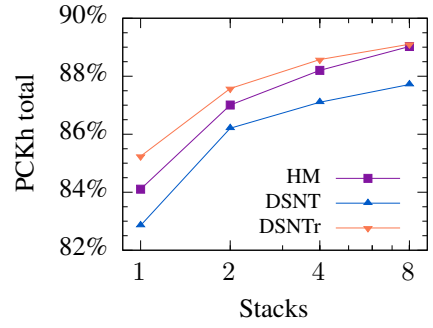


Figure 9: Varying output strategy and stack count for hourglass [4] models.

Fully connected (FC) A softmax heatmap activation is applied to the output of the FCN, followed by a fully connected layer which produces numerical coordinates. The model is trained with Euclidean loss.

DSNT Same as fully connected, but with our DSNT layer instead of the fully connected layer.

DSNT with regularization (DSNTTr) Same as DSNT, but with the inclusion of a regularization term in the loss function. The method of regularization we selected was Jensen-Shannon divergence with $\sigma_t = 1$ and $\lambda = 1$, which empirically performed best.

6.3. Dataset and training

We use the MPII human pose dataset [13] to evaluate the effectiveness of our DSNT layer on an important real-world task. The dataset contains images of 28,883 people with up to 16 joint annotations each, along with approximate person location and scale labels to facilitate the cropping of single-person poses.

Samples from the dataset were augmented during training time using the same scheme as Newell et al. [4], which consists of horizontal flips, 75%-125% scaling, ± 30 degree rotation, and 60%-140% channel-wise pixel value scaling. Since the test set labels are not public, we evaluate on the fixed validation set used in [22] and [4].

The models were optimized with RMSProp [23] using an initial learning rate of 2.5×10^{-4} . Each model was trained for 120 epochs, with the learning rate reduced by a factor of 10 at epochs 60 and 90 (an epoch is one complete pass over the training set). Training was completed on single Maxwell-architecture NVIDIA Titan X GPUs.

Our ResNet-based networks were trained using mini-batches of 32 samples each, with the exception of highly memory-intensive configurations (*e.g.* ResNet-101@28px). The stacked hourglass models were trained using mini-batches of 6 samples each. Our implementation code for

DSNT, written in PyTorch, is available online¹.

6.4. Results

The PCKh performance metric is the percentage of joints with predicted locations that are no further than half of the head segment length from the ground truth. As per the evaluation code provided by MPII, we exclude the pelvis and thorax joints from the average total PCKh.

In order to compare the different approaches to coordinate regression, we conducted a series of experiments with a ResNet-34-based network (Figure 7). The heatmap matching achieved a very low PCKh of 44% at 7×7 px heatmap resolution, which falls outside the bounds of the figure. As the resolution increases, the performance of heatmap matching improves relative to the other approaches, which is evidence of the quantization effects inherent to calculating coordinates via a pixel-wise argmax. This demonstrates that heatmap matching is not suitable for models which generate low-resolution heatmaps, whereas DSNT is largely robust to heatmap size. At higher resolutions, the fully connected approach performs worst. Our DSNT approach exhibits good performance across all resolutions—even 7×7 px—due to the predictions produced by DSNT not having precision dependent on pixel size.

Regularization improves DSNT accuracy in all cases except the lowest resolution, where boundary effects come into play (*i.e.* a 1 pixel standard deviation Gaussian drawn in a 7×7 px image is likely to clip heavily, which adversely affects the DSNT calculation). Fully connected output was found to be worse than heatmap matching at higher resolutions, and worse than DSNT in general.

We conducted further experiments with ResNet-based [12] models to evaluate the impact that depth has on performance. The results in Figure 8 suggest that higher heatmap resolution is beneficial at any depth. However, the trade-off is that increasing resolution with dilations has a large impact on memory consumption and computational cost. For this

¹<https://github.com/anibali/dsntnn>

Table 4: MPII human pose test set PCKh accuracies and inference-time efficiency results.

Method	Head	Shoul.	Elbow	Wrist	Hip	Knee	Ankle	Total	Time (ms)*	Memory*
Tompson et al. [22]	96.1	91.9	83.9	77.8	80.9	72.3	64.8	82.0	-	-
Rafi et al. [24]	97.2	93.9	86.4	81.3	86.8	80.6	73.4	86.3	27.6±0.1	2768 MiB
Wei et al. [25]	97.8	95.0	88.7	84.0	88.4	82.8	79.4	88.5	106.8±0.2	5832 MiB
Bulat et al. [26]	97.9	95.1	89.9	85.3	89.4	85.7	81.7	89.7	41.3±0.2	1432 MiB
Newell et al. [4]	98.2	96.3	91.2	87.1	90.1	87.4	83.6	90.9	60.5±0.1	1229 MiB
Yang et al. [10]	98.5	96.7	92.5	88.7	91.1	88.6	86.0	92.0	194.6±76.8	1476 MiB
DSNTr ResNet-50@28px	97.8	96.0	90.0	84.3	89.8	85.2	79.7	89.5	18.6±0.5	636 MiB

* Any test time data augmentations (horizontal flips, multi-scale) were disabled for time and memory measurements.

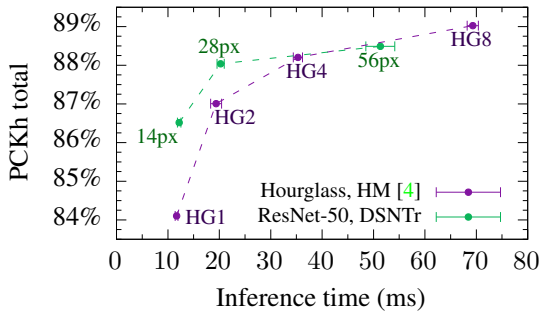


Figure 10: Validation accuracy vs inference time, closer to the top-left is better. Labels show heatmap resolution (ResNet models) or stack count (hourglass models).

reason, we could not train ResNet-101@56px. PCKh was found to increase significantly with depth up until ResNet-50, with only a slight gain observed when increasing the depth even further to ResNet-101.

In addition to ResNet, we also trained stacked hourglass networks [4]. Even though the stacked hourglass architecture was developed using heatmap matching, we found that models trained using DSNT with regularization achieved consistently better results (Figure 9). Analysis of misclassified examples revealed that DSNT was less accurate for predicting edge case joints that lie very close to the image boundary, which is expected due to how the layer works.

Figure 10 directly compares stacked hourglass networks trained with heatmap matching and our ResNet-based networks trained with DSNT and regularization. Although the 8-stack hourglass network was found to have the highest overall accuracy, the ResNet-based models were found to be much faster with only modest concessions in terms of accuracy. For instance, ResNet-50@28px has 8% fewer parameters, requires less than half of the memory during training, and is over $3\times$ faster at inference than HG8, whilst still achieving $\sim 99\%$ of the PCKh score.

Spatial generalization was tested by training models with a restricted training set size and no data augmentation. Fig-

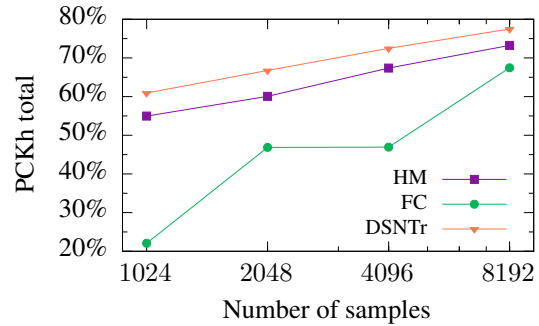


Figure 11: Varying number of training samples (no augmentation) for ResNet-34@28px models.

ure 11 shows that fully connected output exhibits very poor spatial generalization, achieving the extremely low PCKh score of 22% when trained on 1024 samples. On the other hand, both DSNT and heatmap matching perform much better with fewer samples, indicating better generalization.

Finally, we evaluated our ResNet-50@28px DSNTr model on the test set. The results in Table 4 show that our solution, using a much smaller and simpler model (ResNet-50), was able to achieve accuracy competitive with more complex models. A consequence of using a smaller model is that ResNet-50@28px infers significantly faster and uses less memory than all other methods shown in the table. Note that we determined the running time and memory usage of the other methods by downloading pretrained models.

7. Conclusion

There are multiple possible approaches to using CNNs for numerical coordinate regression tasks, each of which affects the behavior of the model in different ways—a fully connected output layer reduces spatial generalization, and heatmap matching introduces issues with differentiability and quantization. In contrast, our proposed DSNT layer can be used to adapt fully convolutional networks for coordinate regression without introducing these problems. We have shown that models built with DSNT can achieve com-

petitive results on real human pose data without complex task-specific architectures, forming a strong baseline. Such models also offer a better accuracy to inference speed trade-off when compared with stacked hourglass models.

Interesting directions for future work are to integrate DSNT with complex pose estimation approaches (e.g. adversarial training [18, 17]), or to use DSNT as an internal layer for models where intermediate coordinate prediction is required (e.g. Spatial Transformer Networks [8]).

References

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” in *Advances in neural information processing systems*, 2012, pp. 1097–1105.
- [2] J. Long, E. Shelhamer, and T. Darrell, “Fully convolutional networks for semantic segmentation,” in *The IEEE Conference on Computer Vision and Pattern Recognition*, 2015, pp. 3431–3440.
- [3] A. Radford, L. Metz, and S. Chintala, “Unsupervised representation learning with deep convolutional generative adversarial networks,” in *International Conference on Learning Representations*, 2016.
- [4] A. Newell, K. Yang, and J. Deng, “Stacked hourglass networks for human pose estimation,” in *European Conference on Computer Vision*. Springer, 2016, pp. 483–499.
- [5] Y. LeCun, B. E. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. E. Hubbard, and L. D. Jackel, “Handwritten digit recognition with a back-propagation network,” in *Advances in neural information processing systems*, 1990, pp. 396–404.
- [6] M. Lin, Q. Chen, and S. Yan, “Network in network,” in *International Conference on Learning Representations*, 2014.
- [7] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning internal representations by error propagation,” DTIC Document, Tech. Rep., 1985.
- [8] M. Jaderberg, K. Simonyan, A. Zisserman *et al.*, “Spatial transformer networks,” in *Advances in Neural Information Processing Systems*, 2015, pp. 2017–2025.
- [9] J. J. Tompson, A. Jain, Y. LeCun, and C. Bregler, “Joint training of a convolutional network and a graphical model for human pose estimation,” in *Advances in neural information processing systems*, 2014, pp. 1799–1807.
- [10] W. Yang, S. Li, W. Ouyang, H. Li, and X. Wang, “Learning feature pyramids for human pose estimation,” in *The IEEE International Conference on Computer Vision*, 2017.
- [11] A. Toshev and C. Szegedy, “DeepPose: Human pose estimation via deep neural networks,” in *The IEEE Conference on Computer Vision and Pattern Recognition*, 2014, pp. 1653–1660.
- [12] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *The IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [13] M. Andriluka, L. Pishchulin, P. Gehler, and B. Schiele, “2d human pose estimation: New benchmark and state of the art analysis,” in *The IEEE Conference on Computer Vision and Pattern Recognition*, 2014, pp. 3686–3693.
- [14] D. C. Luvizon, H. Tabia, and D. Picard, “Human pose regression by combining indirect part detection and contextual information,” *arXiv preprint arXiv:1710.02322*, 2017.
- [15] K. M. Yi, E. Trulls, V. Lepetit, and P. Fua, “Lift: Learned invariant feature transform,” in *European Conference on Computer Vision*. Springer, 2016, pp. 467–483.
- [16] S. Levine, C. Finn, T. Darrell, and P. Abbeel, “End-to-end training of deep visuomotor policies,” *Journal of Machine Learning Research*, vol. 17, no. 39, pp. 1–40, 2016.
- [17] Y. Chen, C. Shen, X.-S. Wei, L. Liu, and J. Yang, “Adversarial posenet: A structure-aware convolutional network for human pose estimation,” in *The IEEE International Conference on Computer Vision*, 2017.
- [18] C.-J. Chou, J.-T. Chien, and H.-T. Chen, “Self adversarial training for human pose estimation,” *arXiv preprint arXiv:1707.02439*, 2017.
- [19] X. Chu, W. Yang, W. Ouyang, C. Ma, A. L. Yuille, and X. Wang, “Multi-context attention for human pose estimation,” in *The IEEE Conference on Computer Vision and Pattern Recognition*, 2017.
- [20] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial nets,” in *Advances in neural information processing systems*, 2014, pp. 2672–2680.
- [21] F. Yu and V. Koltun, “Multi-scale context aggregation by dilated convolutions,” in *ICLR*, 2016.
- [22] J. Tompson, R. Goroshin, A. Jain, Y. LeCun, and C. Bregler, “Efficient object localization using convolutional networks,” in *The IEEE Conference on Computer Vision and Pattern Recognition*, 2015, pp. 648–656.

- [23] T. Tieleman and G. Hinton, “Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude,” *COURSERA: Neural networks for machine learning*, vol. 4, no. 2, 2012.
- [24] U. Rafi, B. Leibe, J. Gall, and I. Kostrikov, “An efficient convolutional network for human pose estimation.” in *BMVC*, vol. 1, 2016, p. 2.
- [25] S.-E. Wei, V. Ramakrishna, T. Kanade, and Y. Sheikh, “Convolutional pose machines,” in *The IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 4724–4732.
- [26] A. Bulat and G. Tzimiropoulos, “Human pose estimation via convolutional part heatmap regression,” in *European Conference on Computer Vision*. Springer, 2016, pp. 717–732.